

```

import numpy as np

def gram_schmidt(V):
    # Ortonormalisasi kolom-kolom dari matriks V
    U = []
    for v in V.T:
        for u in U:
            v = v - np.dot(u, v) * u
        v = v / np.linalg.norm(v)
        U.append(v)
    return np.array(U).T # Kembalikan sebagai matriks kolom

def diagonalisasi_ortogonal(A):
    print("Input matrix A:\n", A)

    # Step 1: Hitung nilai eigen dan vektor eigen
    eigenvalues, eigenvectors = np.linalg.eig(A)

    print("\nNilai eigen ( $\lambda$ ):\n", eigenvalues)
    print("\nVektor eigen (sebelum ortonormalisasi):\n", eigenvectors)

    # Step 2: Ortonormalisasi vektor eigen (manual Gram-Schmidt)
    Q = gram_schmidt(eigenvectors)
    print("\nMatriks P (vektor eigen ortonormal sebagai kolom):\n", Q)

    # Step 3: Hitung  $P^T A P$ 
    D = Q.T @ A @ Q
    print("\nHasil  $P^T A P$  (matriks diagonal D):\n", D)

    return eigenvalues, Q, D

# Contoh penggunaan
if __name__ == "__main__":
    A = np.array([
        [3, 1, 1],
        [1, 3, 1],
        [1, 1, 3]
    ])

    eigvals, P, D = diagonalisasi_ortogonal(A)

```

HASIL/OUTPUT:

Input matrix A:

```
[[3 1 1]
 [1 3 1]
 [1 1 3]]
```

Nilai eigen (λ):

```
[2. 5. 2.]
```

Vektor eigen (sebelum ortonormalisasi):

```
[[-0.81649658  0.57735027 -0.29329423]
 [ 0.40824829  0.57735027 -0.5132649 ]
 [ 0.40824829  0.57735027  0.80655913]]
```

Matriks P (vektor eigen ortonormal sebagai kolom):

```
[[-8.16496581e-01  5.77350269e-01 -5.02793658e-17]
 [ 4.08248290e-01  5.77350269e-01 -7.07106781e-01]
 [ 4.08248290e-01  5.77350269e-01  7.07106781e-01]]
```

Hasil $P^T A P$ (matriks diagonal D):

```
[[2.00000000e+00 1.16573418e-15 1.11022302e-16]
 [8.88178420e-16 5.00000000e+00 0.00000000e+00]
 [2.22044605e-16 0.00000000e+00 2.00000000e+00]]
```

**** Process exited - Return Code: 0 ****

Press Enter to exit terminal